Name : Vaibhav Gupta
Div : D15C
Batch : C
Roll No : 61

MLDL Experiment 01

Aim: Implement Linear and Logistic Regression on real-world datasets.

Linear Regression:

1. Dataset Source:

- **Dataset Name**: House Price Prediction Dataset
- **Source:** https://www.kaggle.com/datasets/suvidyasonawane/house-price-prediction-dataset
- **Repository Owner:** Suvidya Sonawane

2. Dataset Description:

This dataset contains 300 synthetic housing records created for practicing regression and machine learning workflows.
The data simulates realistic relationships between housing features and sale prices, making it ideal for exploratory data analysis (EDA), feature engineering, and regression modeling.

- Size: 300 samples (rows) × multiple numerical attributes (columns)
- Target Variable:

0 SalePrice (Continuous) – Final selling price of the house

- Key Features:

1. OverallQual – Overall material and finish quality of the house
2. GrLivArea – Above ground living area (square feet)
3. GarageCars – Number of cars that can fit in the garage
4. TotalBsmtSF – Total basement area (square feet)
5. YearBuilt – Year the house was constructed
6. TotRmsAbvGrd – Total rooms above ground
7. LotArea – Size of the lot in square feet

The dataset is clean, beginner-friendly, and requires no preprocessing, allowing direct application of regression algorithms.

3. <u>Mathematical Formulation of the Algorithm:</u>

Multiple Linear Regression models the relationship between several independent variables and a continuous dependent variable.

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

Where:

- $\hat{y}$ = Predicted house price
- β_0 = Intercept
- $\beta_1 \ldots \beta_n$ = Coefficients
- $x_1 \ldots x_n$ = Input features

Cost Function (Mean Squared Error):

$$J(\beta) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

4. <u>Algorithm Limitations:</u>

1. **Linearity Assumption:** Assumes linear relationship between features and price.

2. **Sensitive to Outliers:** Extremely expensive houses may distort predictions.

3. **Multicollinearity:** Highly correlated features can reduce model stability.

5. <u>Methodology / Workflow:</u>

1. Load CSV dataset

2. Select input features and target (SalePrice)

3. Split dataset into training and testing sets (80:20)
4. Train Multiple Linear Regression model
5. Predict house prices
6. Evaluate using MSE and R^2 score

6. Performance Analysis:

- **Metric 1:** Mean Squared Error (MSE)
    - Measures average squared prediction error
- **Metric 2:** R-Squared (R^2)
    - Indicates percentage of variance in house prices explained by features

Higher R^2 and lower MSE indicate better model performance.

7. Hyperparameter Tuning:

Standard Linear Regression has no tunable hyperparameters.
To improve performance, **Ridge Regression (L2 Regularization)** can be applied.

- Tuned Parameter: `alpha` ● Tested Values: [0.1, 1, 10]
- Helps reduce overfitting and improve generalization.

Logistic Regression:

1. Dataset Source:

- **Dataset Name:** Weather Forecast Dataset

- **Source:** https://www.kaggle.com/datasets/zeeshier/weather-forecastdataset

- **Repository Owner:** Zeeshier

2. Dataset Description:

This dataset contains **2,500 weather observations** and is designed for beginners to practice **classification using machine learning**.
The dataset is used to predict whether **rainfall will occur** based on several atmospheric and environmental conditions.

- **Size:** 2,500 samples (rows) × multiple attributes (columns)

- **Target Variable:**

  0 Rain (Binary)

    - 1 → Rainfall occurs

    - 0 → No rainfall

- **Key Features:**

1. Temperature – Current temperature

2. Humidity – Moisture content in air

3. WindSpeed – Speed of wind

4. Pressure – Atmospheric pressure

5. CloudCover – Percentage of cloud cover

6. Visibility – Distance that can be clearly seen

7. DewPoint – Temperature at which condensation occurs

The dataset is **clean, simple, and beginner-friendly**, making it suitable for experimenting with classification algorithms such as Logistic Regression, Decision Trees, and Random Forests.

3. <u>Mathematical Formulation of the Algorithm:</u>

Logistic Regression estimates the probability that an instance belongs to a particular class (Rain or No Rain).

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Where:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

Decision Rule:

$$\hat{y} = \begin{cases} 1, & \text{if } \sigma(z) \geq 0.5 \\ 0, & \text{if } \sigma(z) < 0.5 \end{cases}$$

4. Algorithm Limitations:

1. **Linear Decision Boundary:** Cannot model complex non-linear relationships.

2. **Sensitive to Feature Scaling:** Features should be standardized.

3. **Performance Drops with Highly Correlated Features.**

5. Methodology / Workflow:

1. Load weather dataset

2. Separate features and target (Rain)

3. Apply feature scaling (StandardScaler)

4. Split dataset into training and testing sets (80:20)

5. Train Logistic Regression model

6. Predict rainfall

7. Evaluate performance

6. Performance Analysis:

- **Metric 1: Accuracy**

    0 Measures overall correctness of predictions

- **Metric 2: Confusion Matrix**

    0 Shows True Positives, False Positives, True Negatives, False Negatives

Higher accuracy and balanced confusion matrix indicate better model performance.

7. <u>Hyperparameter Tuning:</u>

- Parameter Tuned: `C` (Inverse of Regularization Strength)
- Tested Values: [0.01, 0.1, 1, 10, 100]

Impact:

- Low C → Strong regularization (simpler model)
- High C → Weak regularization (complex model)

Code And Output:

```
from google.colab import
files

import pandas as pd

import numpy as np

import matplotlib.pyplot
as plt

from
sklearn.model_selection
import train_test_split

from sklearn.linear_model
import LinearRegression

from sklearn import
metrics



uploaded = files.upload()

df =
pd.read_csv(list(uploaded.
keys())[0])



X = df.drop('SalePrice',
axis=1)

y = df['SalePrice']



X =
X.select_dtypes(include=[n
p.number])

X_train, X_test, y_train,
y_test =
train_test_split(X,y,test_
size=0.3,random_state=42)



model = LinearRegression()

model.fit(X_train,y_train)



y_pred =
model.predict(X_test)



mae =
metrics.mean_absolute_erro
r(y_test,y_pred)

mse =
metrics.mean_squared_error
(y_test,y_pred)

rmse = np.sqrt(mse)

r2 =
metrics.r2_score(y_test,y_
pred)



print("MAE:",mae)

print("MSE:",mse)

print("RMSE:",rmse)

print("R2:",r2)
```

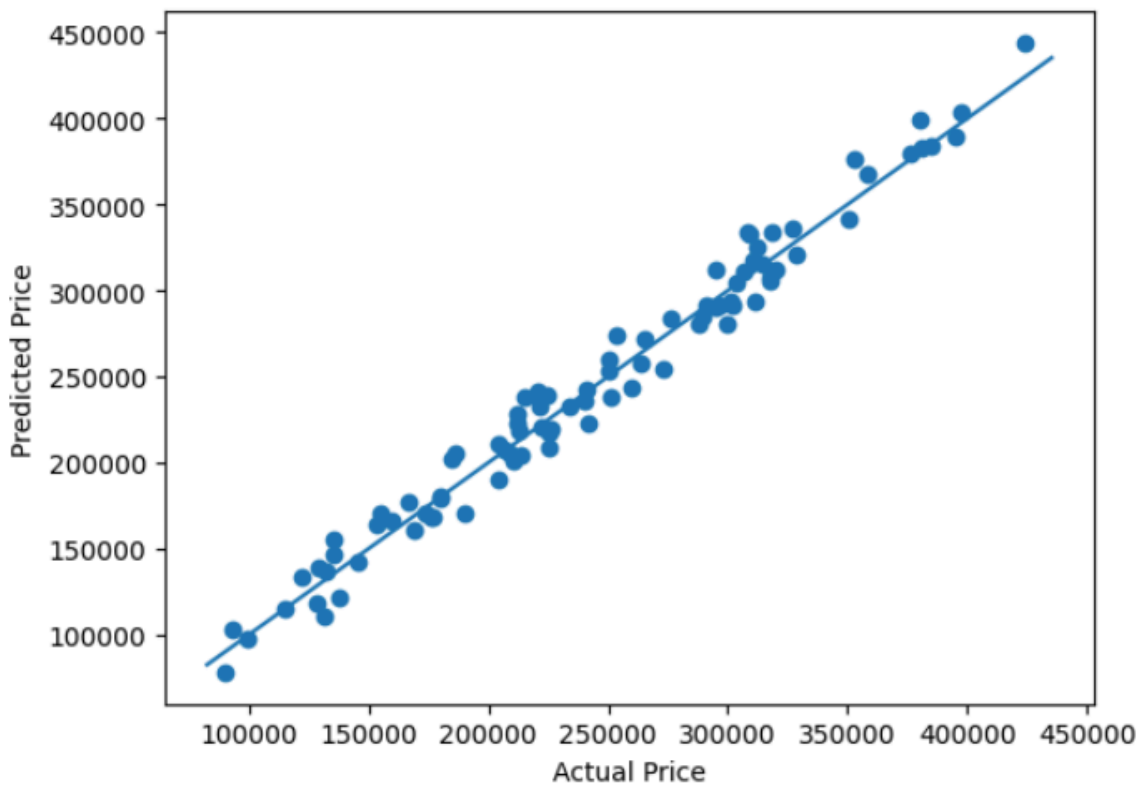
```
plt.scatter(y_test,y_pred)

plt.plot([y.min(),y.max()]
,[y.min(),y.max()])

plt.xlabel("Actual Price")
```

```
plt.ylabel("Predicted
Price")

plt.show()
```

Choose Files house_pric...practice.csv

house_prices_practice.csv(text/csv) - 11712 bytes, last modified: 3/17/2026 - 100% done

```
Saving house_prices_practice.csv to house_prices_practice.csv
MAE: 10126.1733282958
MSE: 144540866.0003639
RMSE: 12022.514961536288
R2: 0.976991889057285
```



```
from google.colab import
files

import pandas as pd

import numpy as np

import matplotlib.pyplot
as plt

from
sklearn.model_selection
import
train_test_split,GridSearc
hCV

from sklearn.linear_model
import Ridge

from sklearn import
metrics



uploaded = files.upload()

df =
pd.read_csv(list(uploaded.
keys())[0])



X = df.drop('SalePrice',
axis=1)

y = df['SalePrice']



X =
X.select_dtypes(include=[n
p.number])

X_train,X_test,y_train,y_t
est =
train_test_split(X,y,test_
size=0.3,random_state=42)



params =
{'alpha':[0.1,1,10]}



ridge = Ridge()

grid =
GridSearchCV(ridge,params,
cv=5)



grid.fit(X_train,y_train)



best_model =
grid.best_estimator_



y_pred =
best_model.predict(X_test)



mae =
metrics.mean_absolute_erro
r(y_test,y_pred)

mse =
metrics.mean_squared_error
(y_test,y_pred)

rmse = np.sqrt(mse)
```

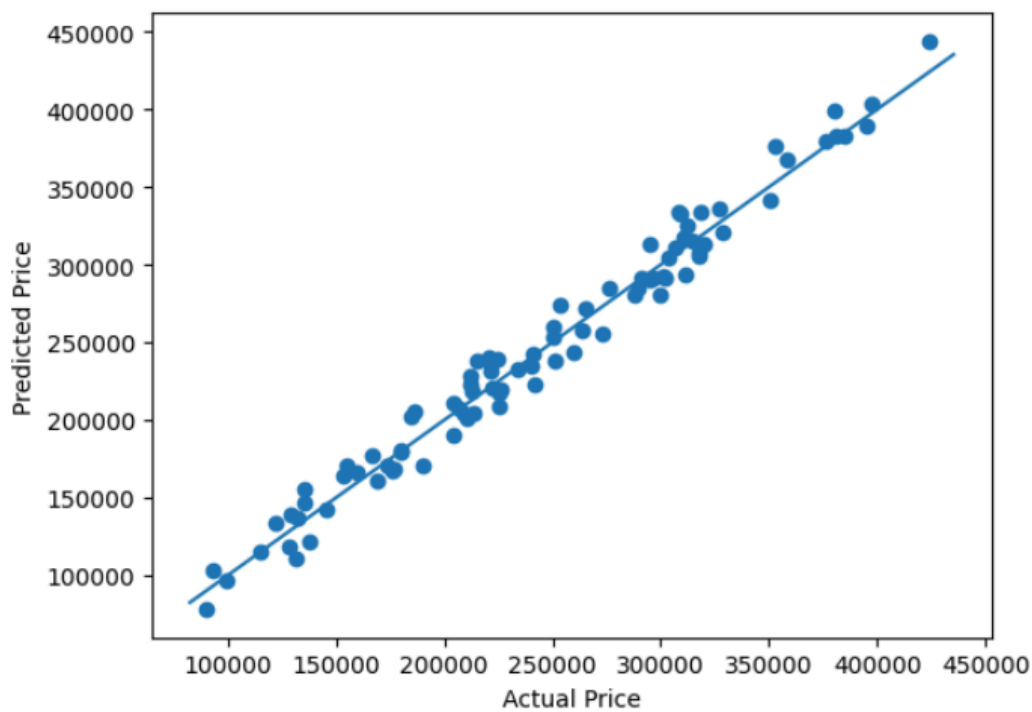
```
r2 =
metrics.r2_score(y_test,y_
pred)



print("Best
Alpha:",grid.best_params_)

print("MAE:",mae)

print("MSE:",mse)

print("RMSE:",rmse)

print("R2:",r2)
```

```
plt.scatter(y_test,y_pred)

plt.plot([y.min(),y.max()]
,[y.min(),y.max()])

plt.xlabel("Actual Price")

plt.ylabel("Predicted
Price")

plt.show()
```

Choose Files house_pric...practice.csv

house_prices_practice.csv(text/csv) - 11712 bytes, last modified: 3/17/2026 - 100% done

```
Saving house_prices_practice.csv to house_prices_practice (1).csv
Best Alpha: {'alpha': 1}
MAE: 10134.632577561504
MSE: 144362780.6434515
RMSE: 12015.106351732868
R2: 0.9770202368025455
```



```
from google.colab import files

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import
train_test_split

from sklearn.preprocessing import
StandardScaler

from sklearn.linear_model import
LogisticRegression

from sklearn.metrics import
accuracy_score,confusion_matrix



uploaded = files.upload()

df =
pd.read_csv(list(uploaded.keys())[0]
)



target = df.columns[-1]



if df[target].dtype == object:

    df[target] =
df[target].astype('category').cat.co
des



X = df.drop(target,axis=1)

y = df[target]
```

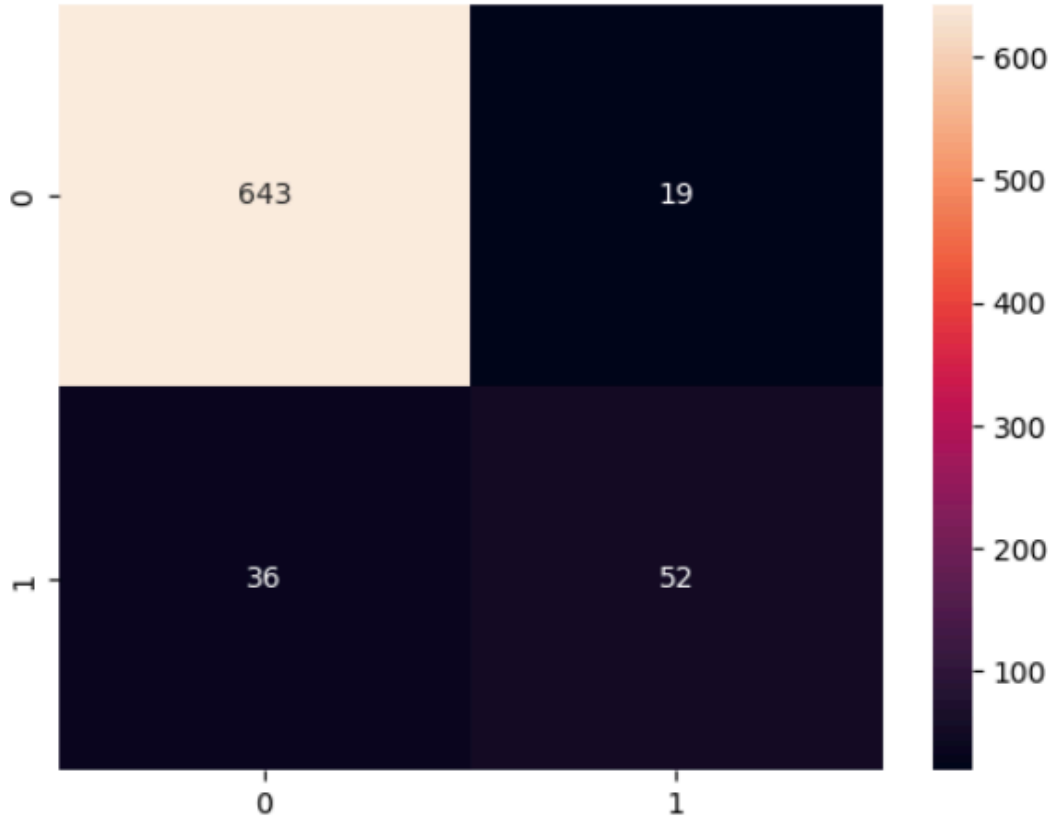
```
X =
X.select_dtypes(include=[np.number])


X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.3,r
andom_state=42)


scaler = StandardScaler()


X_train =
scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)


model =
LogisticRegression(max_iter=10000)


model.fit(X_train,y_train)


y_pred = model.predict(X_test)


acc = accuracy_score(y_test,y_pred)


print("Accuracy:",acc)


cm = confusion_matrix(y_test,y_pred)
```

```
sns.heatmap(cm,annot=True,fmt='d')
plt.show()
```

Choose Files weather_for...st_data.csv

weather_forecast_data.csv(text/csv) - 251223 bytes, last modified: 3/17/2026 - 100% done

```
Saving weather_forecast_data.csv to weather_forecast_data.csv
Accuracy: 0.9266666666666666
```



```
from google.colab import files

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import
train_test_split,GridSearchCV

from sklearn.preprocessing import
StandardScaler

from sklearn.linear_model import
LogisticRegression

from sklearn.metrics import
accuracy_score,confusion_matrix


uploaded = files.upload()

df =
pd.read_csv(list(uploaded.keys())[0]
)


target = df.columns[-1]


if df[target].dtype == object:

    df[target] =
df[target].astype('category').cat.co
des


X = df.drop(target,axis=1)

y = df[target]


X =
X.select_dtypes(include=[np.number])


X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.3,r
andom_state=42)


scaler = StandardScaler()


X_train =
scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)


params = {'C':[0.01,0.1,1,10,100]}


model =
LogisticRegression(max_iter=10000)


grid =
GridSearchCV(model,params,cv=5)


grid.fit(X_train,y_train)


best_model = grid.best_estimator_


y_pred = best_model.predict(X_test)
```

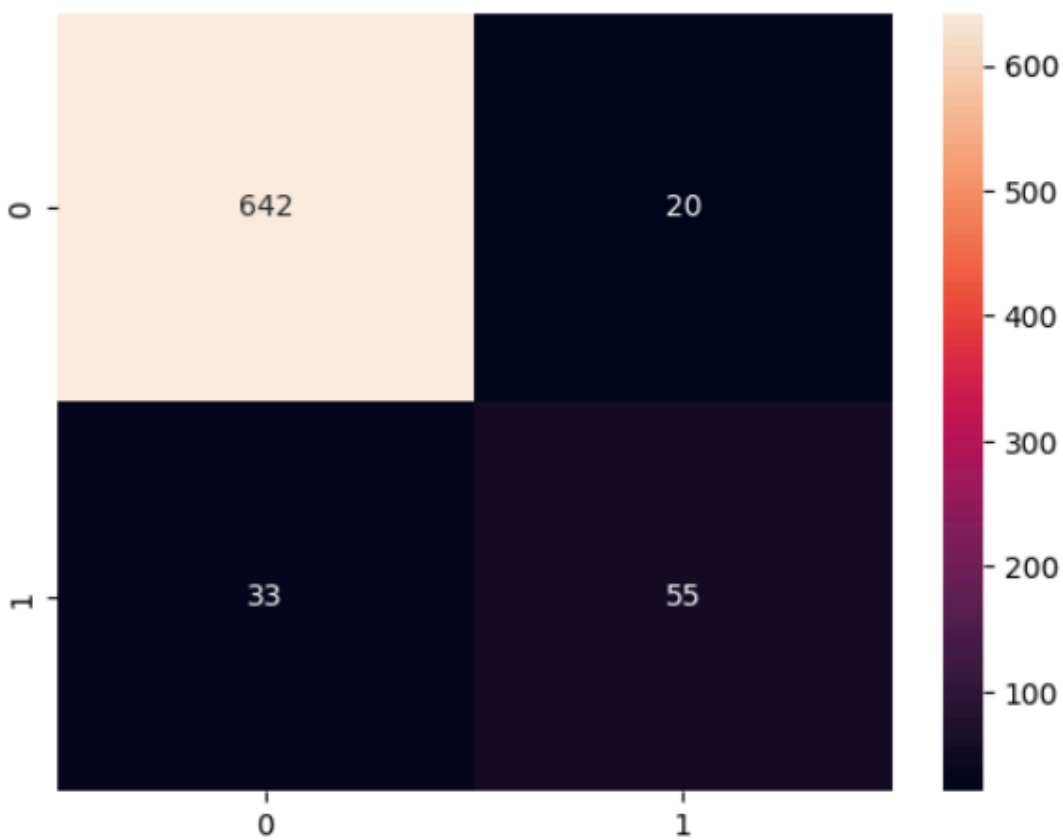
```
acc = accuracy_score(y_test,y_pred)

print("Best C:",grid.best_params_)

print("Accuracy:",acc)
```

```
cm = confusion_matrix(y_test,y_pred)

sns.heatmap(cm,annot=True,fmt='d')

plt.show()
```

Choose Files weather_for...st_data.csv

weather_forecast_data.csv(text/csv) - 251223 bytes, last modified: 3/17/2026 - 100% done

```
Saving weather_forecast_data.csv to weather_forecast_data (1).csv
Best C: {'C': 100}
Accuracy: 0.9293333333333333
```



Interpretation:

Linear Regression

The graph represents the relationship between the actual house prices and the predicted prices generated by the Linear Regression model.

- The X-axis represents the actual sale prices from the test dataset.
- The Y-axis represents the predicted prices produced by the model.
- The diagonal line represents the ideal prediction line, where predicted values perfectly match the actual values.

Most of the data points are clustered close to the diagonal line, indicating that the predicted prices are very close to the actual house prices.

This suggests that the Linear Regression model has learned the relationship between housing features and sale price effectively.

A high R^2 value (around 0.97) indicates that the model explains approximately 97% of the variance in house prices, showing strong predictive performance.

However, a few points slightly away from the line indicate minor prediction errors, which may occur due to feature limitations or natural variation in housing prices.

Logistic Regression:

The confusion matrix visualizes the classification performance of the Logistic Regression model.

The matrix contains four values:

- True Positives (TP): Correctly predicted rainfall cases

- True Negatives (TN): Correctly predicted no-rain cases
- False Positives (FP): Predicted rain when there was no rain
- False Negatives (FN): Predicted no rain when rain actually occurred

From the matrix values in the experiment:

- TN = 643
- FP = 19
- FN = 36
- TP = 52

This indicates that the model correctly classifies most cases, especially no-rain conditions, which dominate the dataset.

The relatively low number of false predictions suggests that the model performs well in distinguishing between rain and no-rain scenarios.

Conclusion:

This experiment demonstrated the practical implementation of Linear Regression for house price prediction and Logistic Regression for rainfall prediction using Kaggle datasets. Both models successfully captured relationships between input features and their respective target variables, showing the effectiveness of supervised learning techniques for solving real-world problems.

The experiment highlighted the importance of data preprocessing, feature selection, feature scaling, and train–test splitting in building reliable machine learning models. Performance evaluation using metrics such as Mean Squared Error, R^2 score, Accuracy, and Confusion Matrix provided insights into model effectiveness. Overall, this experiment strengthened understanding of regression and classification algorithms and their application in predictive analytics.